

Full DouZero+: Improving *DouDizhu* AI by Opponent Modeling, Coach-Guided Training and Bidding Learning

Youpeng Zhao¹, Graduate Student Member, IEEE, Jian Zhao², Xunhan Hu²,
Wengang Zhou³, Senior Member, IEEE, and Houqiang Li³, Fellow, IEEE

Abstract—With the development of deep reinforcement learning, much progress in various perfect and imperfect information games has been achieved. Among these games, *DouDizhu*, a popular card game in China, poses great challenges because of the imperfect information, large state and action space as well as the cooperation issue. In this article, we put forward an AI system for this game, which adopts opponent modeling and coach-guided training to help agents make better decisions when playing cards. Besides, we take the bidding phase of *DouDizhu* into consideration, which is usually ignored by existing works, and train a bidding network using Monte Carlo simulation. As a result, we achieve a full version of our AI system that is applicable to real-world competitions. We conduct extensive experiments to evaluate the effectiveness of the three techniques adopted in our method and demonstrate the superior performance of our AI over the state-of-the-art *DouDizhu* AI, i.e., DouZero. We upload our AI systems, one is bidding-free and the other is equipped with a bidding network, to Botzone platform and they both rank the first among over 400 and 250 AI programs on the two corresponding leaderboards, respectively.

Index Terms—Coach-guided training, *DouDizhu*, Monte Carlo (MC) method, Monte Carlo simulation, opponent modeling, reinforcement learning (RL).

I. INTRODUCTION

RECENT years have witnessed the great advance of game AI development in the artificial intelligence field [1], [2],

Manuscript received 16 November 2022; revised 21 March 2023, 6 June 2023, and 18 July 2023; accepted 19 July 2023. Date of publication 28 July 2023; date of current version 17 September 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61836011, and in part by the GPU cluster built by MCC Lab of Information Science and Technology Institution, USTC, and the Supercomputing Center of the USTC. Recommended by Associate Editor R. Thawonmas. (Corresponding authors: Wengang Zhou; Houqiang Li.)

Youpeng Zhao, Jian Zhao, and Xunhan Hu are with the CAS Key Laboratory of Technology in Geo-spatial Information Processing and Application System, Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei 230027, China (e-mail: zyp123@mail.ustc.edu.cn; zj140@mail.ustc.edu.cn; cathy-hxh@mail.ustc.edu.cn).

Wengang Zhou and Houqiang Li are with the CAS Key Laboratory of Technology in Geo-spatial Information Processing and Application System, Department of Electronic Engineering and Information Science, University of Science and Technology of China, Hefei 230027, China, and also with the Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, Hefei 230027, China (e-mail: zhwg@ustc.edu.cn; lihq@ustc.edu.cn).

Our codes are available at https://github.com/submit-paper/Doudizhu_plus.
Digital Object Identifier 10.1109/TG.2023.3299612

[3], [4]. AI programs of human-level or superhuman level have been developed in various games, including board game, such as *Go* [1], [5] and chess [6], card games, such as *Limit/No-Limit Texas Hold'em* [7], [8], [9] and *Mahjong* [10], and video games, such as *StarCraft* [11], *DOTA* [12], and *Honor of Kings* [13]. Whereas, how to handle imperfect-information games with large state and action space is still very challenging and waiting to be studied.

In this article, we aim to develop an AI program for *DouDizhu*, a.k.a, *Fighting the Landlord*, which is the most popular card game in China and has not been studied profoundly until recently. Generally, a complete *DouDizhu* game includes two phases, bidding and card-playing. In the bidding phase, the three players have to bid for the Landlord based on their initial hand cards and the Landlord can gain the Landlord cards. A player has to bet a higher stake than the previous one, otherwise he has to pass his turn. In the card-playing phase, the players need to make appropriate decisions to play cards, with the objective of emptying their cards. Fig. 1 shows examples for these phases.

Some characteristics of *DouDizhu* make this game very challenging. First, unlike perfect-information games where there is no hidden game information for each agent, *DouDizhu* is an imperfect-information game where players do not know the initial hand cards of others. Second, this game involves both cooperation and competition simultaneously. To be specific, the two Peasants act as a team to play against the Landlord. A typical case is demonstrated in Fig. 1(b) where the Peasant at the bottom chooses to play a small Solo card for the winning of his partner. This property makes this game interesting but also complex, which may fail classical algorithms for poker games, such as counterfactual regret minimization [14] and its variants. Third, this game possesses very large state space and action space owing to the combinations of cards and its complex rules compared with other card games. The information set size is up to 10^{24} , which is much larger than *Heads-Up Limit/No-Limit Texas Hold'em* and *1v1 Mahjong* [15], and there are more than 10^4 possible actions in the action space.

In spite of the issues mentioned above, some achievements have been made in building AI programs for *DouDizhu* by introducing heuristic rules to the learning of the model or game

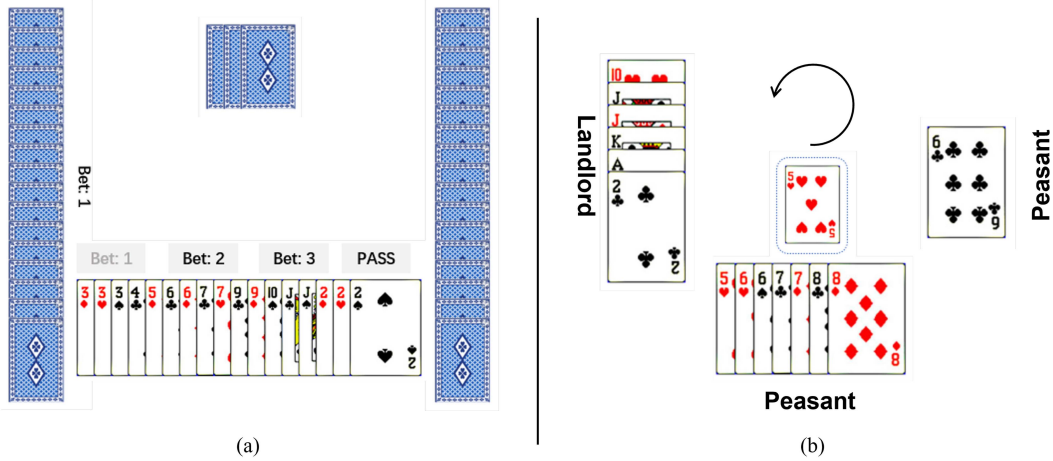


Fig. 1. Examples of *DouDizhu* game. (a) Example of the bidding phase. A player has to bet a higher stake than the previous one, otherwise he has to pass. (b) Demonstrates a case example of cooperation in *DouDizhu*. If the Peasants learn to cooperate with each other, current player should play a small Solo to let the teammate to win the game.

tree traversal [16], [17]. Although these works deal with the challenges in *DouDizhu*, they are either far from optimal or computationally expensive as they tend to highly rely on human prior knowledge, which has great effect on their performance. What's more, existing works [16], [17] about *DouDizhu* only considers the card-playing phase as the logic of bidding phase and card-playing phase is so different that it is hard to train a model, which is capable of handling these two phases concurrently.

Recently, DouZero [18] has attracted considerable attention owing to its outstanding performance. It combines classical Monte Carlo (MC) method [19] with deep neural networks to tackle the large state and action space and adopt self-play reinforcement learning (RL) paradigm to accelerate the training. Whereas, even if DouZero has achieved preponderance over previous works that partly rely on heuristic rules, it takes a lot of time for the training. In addition, although DouZero mentions that a model is developed to deal with the bidding phase, it utilizes large amount of human expert data, which makes it impractical to implement this method due to the great difficulty in accessing enough human data.

In this work, we put forward a new framework to develop a stronger *DouDizhu* AI system, which contains three novelties. First, we introduce opponent modeling to predict the hand cards of other players and concatenate the predicted results with original input features for the model to make decision, thus improving the interpretability and decision-making capability of AI. Second, we propose coach-guided training to filter out games where the strength of initial cards is unbalanced so that these comparatively valueless data will not be used for rollout and training, bringing the increase of training efficiency. Third, we add a bidding network, which is trained based on MC simulation and realize the bidding module without human data for the first time, leading to full Douzero: *DouDizhu* AI free of human data.

To demonstrate the strength of our *DouDizhu* AI system, we conduct extensive experiments to compare our method with DouZero and our AI system proves to achieve a better performance. Also, we upload our system to the Botzone [20], [21], [22], [23] platform, which supports leaderboards for *DouDizhu*

games and many users upload their AI to compete with others. This platform support two versions of this game, i.e., *Fight The Landlord* and *Fight The Landlord 2*. The former one only contains the card-playing phase while the other comprises both the bidding phase and card-playing phase and each version maintains its own leaderboard. We upload our AI systems to *Fight The Landlord* and *Fight The Landlord 2* on the platform, respectively, and they both rank the first, among more 400 and 250 AI programs.

The preliminary version of this work is published in [24]. We have not made significant improvements and extensions to our preliminary work. The major technical improvement is that we design a bidding network based on MC simulation so that our AI bot can fit real-world competitions where the bidding phase has to be considered. What is more, we conduct extensive experiments to illustrate the improvements that we propose in our work more clearly. Besides, we change some settings in our preliminary work to make the proposed method more suitable and show the final performance of our AI system compared with DouZero, proving the effectiveness of our method.

II. RELATED WORK

In this section, we describe existing works about *DouDizhu* and then introduce some works about techniques that we adopt in our work, i.e., opponent modeling and MC simulation.

A. RL for *DouDizhu* Game

Due to the notable game complexity of *DouDizhu*, some research efforts build AI programs by combining RL with other techniques. A method called combinatorial Q-learning [16] is proposed to decouple actions into decomposition selection and final move selection. However, this processing approach depends on human prior knowledge, thus limiting its performance. DeltaDou [17] is the first program that manages to reach human-level performance. It integrates fictitious play Monte Carlo tree search (MCTS), which is a variant of MCTS that uses neural network to store the policy, and a policy-based inference algorithm [25] into an AlphaZero-like framework.

However, DeltaDou requires to pretrain a kicker network depending on heuristic rules to abstract the action space, which plays an important role in the performance of AI program. If the selection of the kicker is not optimal, the strength of the bot may be affected. Moreover, the search and inference are so computationally expensive that the training process takes a lot of time.

Recently, DouZero [18] adopts MC methods [19] combined with neural networks to deal with this game and manages to defeat all the previous *DouDizhu* AI programs. Similar technique is also used in some other game AIs, such as a modern board game, *Kingdomino* [26], and a kind of chess, *Tibetan Jiuqi* [27]. The notable performance of DouZero reveals applicability of MC methods to complex imperfect information games, providing new insight into future research.

B. Opponent Modeling for Games

In human practice, knowing some hidden information of opponent will give the player a clear advantage. As a consequence, opponent modeling has attracted substantial attention in game AI. For instance, this technique can be applied in poker games to infer a posterior over opponent's strategies and thus make an appropriate response to that distribution [28]. Recently, inspired by the success of RL, a lot of research efforts have combined RL with opponent modeling. Such combination helps the agent predict opponents' actions and proves to achieve superior performance than the original RL algorithms in some games [29], [30]. In addition, opponent modeling can be applied in multi-agent RL problems in which agents are designed to consider the learning of other agents when updating their own policies [31], [32]. To sum up, existing works demonstrate that introducing opponent modeling to the development of game AI is of great help, especially imperfect information games.

Inspired by the benefits of opponent modeling, we utilize this technique in our *DouDizhu* AI system. Different from above works, which try to model the behavior pattern of opponents, opponent modeling is used to predict the hand cards of other players in this work. Previous works tend to make explicit prediction about opponents' action because the games they deal with are comparatively simple. Whereas, in a complex game, such as *DouDizhu*, it is difficult to make accurate predictions about behavior patterns of opponents so that our practice allows the model to merely analyze known information to predict cards of others. To the best of the authors knowledge, our work is the first to adopt opponent modeling in *DouDizhu* game.

C. MC Simulation for Games

MC simulation is a kind of simulation that relies on repeated sampling and statistical analysis to simulate real physical scenarios and get the approximate solution of a problem. [33]. This technique is based on the law of large numbers so that the statistical error can be made arbitrarily small. As a result, this approach can be adopted in environments that involve hidden information or randomness and it has been researched well [34], [35]. Even in a complex multiplayer game with imperfect information, i.e., a Taiwanese variant of *Mahjong*, with imperfect

information, MC simulation shows a good performance [36]. What is more, this method can also be combined with many other methods, such as opponent modeling, search, and RL. For instance, a computer *Mahjong* player is built using opponent modeling and MC simulation and the bot is claimed to achieve a performance higher than average human player [37]. In addition, after combined with search and RL, these techniques can also be applied in computer fighting games [3].

From the above introduction, MC simulation can be adopted in various fields and combined with many different methods. In our work, we also make use of this technique in the training of the bidding network. In the bidding phase, the agents need to decide whether to bid for Landlord and how much stake to bet, thus affecting the following card-playing phase to some degree. Whereas, it is difficult to clarify how the decision in bidding phase determines the final result of a game. In this way, we consider the two phases separately and just utilize MC simulation to evaluate the strength of initial cards for a player, thus helping the agent to make an appropriate bidding.

III. PRELIMINARY

In this section, we briefly introduce the basic rules of *DouDizhu* for a better understanding of this game. Then, we discuss the main algorithm used in DouZero, i.e., deep Monte Carlo (DMC), which utilizes deep neural networks for approximation in MC methods, and some details of DouZero system.

A. Basic Rules of DouDizhu

DouDizhu, a.k.a. *Fighting the Landlord*, is three-player card game that attracts millions of players in China. This game is easy to learn but difficult to master as it requires careful planning and considerable thinking when making every decision. Among the three players in the game, two of them are called Peasants and they need to cooperate together against the other one, named Landlord. The objective of this game is to be the first player with no cards left in hand. Whereas, if one Peasant has emptied his cards, the whole team can win the game so that a Peasant player can choose to contribute for his partner's winning.

A standard *DouDizhu* game consists of two phases, i.e., bidding phase and card-playing phase. At the beginning of this game, a shuffled pack of cards is dealt to three players and each one is dealt 17 cards, with the remaining three cards left for the Landlord, whom is decided in the bidding phase. In the bidding phase, players need to analyze the strength of their cards and decide whether to bid for the Landlord. If no one is willing to bid, a new pack of cards will be dealt. To be mentioned, the three Landlord cards are kept face down and will be seen by all players after the Landlord has been determined. The player who outbids will become the Landlord and get the three Landlord cards.

After the bidding phase, the players will play cards in turn in counterclockwise order and the first trick starts from the Landlord. The first player can play any category of cards and the other players have to play cards of the same type or bombs, which are larger than the cards played by the previous player, or they can choose to pass. A trick continues until all the other

players pass in succession and the player who played the last cards leads in the next trick. *DouDizhu* is played with one pack of cards, including two jokers and the suits of the cards do not matter in this game. The cards are ranked by red Joker, black Joker, 2, A, K, Q, J, 10, 9, 8, 7, 6, 5, 4, and 3.

One property that makes *DouDizhu* challenging is the rich categories of card combinations. For some categories, the player can choose a kicker card, which can be any card in hand. If an inappropriate kicker card is played, the subsequent decisions are bounded to be affected. As for specific categories of cards and more detailed rules of this game, please refer to Wikipedia¹ for more introduction.

B. Deep Monte Carlo

As a classical method in RL, MC method learns value functions and optimal policies from experience, namely, data of states, actions, and rewards sampled from actual or simulated interactions with the environment [19]. Thanks to the development of neural networks, deep neural networks are usually used for function approximation in the MC method considering its powerful representational capability, leading to the application of DMC in nowadays research. This technique is designed for episodic tasks, where experience can be divided into episodes with finite length, and it updates the value estimation and policy only when an episode terminates. Specifically, after each episode, the observed returns are utilized for policy evaluation and the policy can be improved at the visited states in the episode. The optimization procedure of a policy π in DMC is intuitively described as follows.

- 1) Interact with the environment for an episode using π and get the return according to the result of this episode.
- 2) For each state-action pair (s, a) visited in the episode, calculate and update $Q(s, a)$ with the average return. To be specific, the Q-value is calculated by a network and it is optimized with mean square error (mse) loss.
- 3) For each state s in the episode, a new policy is obtained: $\pi(s) \leftarrow \operatorname{argmax}_{a \in A} Q(s, a)$.

When putting MC methods into practice, an epsilon-greedy policy is usually adopted to balance exploration and exploitation in Step 1. The average return in Step 2 is often achieved by the discounted cumulative reward and the target Q-value is approximated without bias, different from Q-learning, which depends on bootstrapping.

As *DouDizhu* is a typical episodic task, MC is naturally suitable for this game. Although DMC requires a large number of experience for training, it is easy to generate samples efficiently, which can also alleviate the issue of variance. What is more, applying DMC to *DouDizhu* also has some clear advantages compared with other RL algorithms. To be specific, MC methods can avoid the overestimating issue suffered by deep Q-learning and can leverage action features to help learning while traditional policy gradient algorithms usually fail to utilize these knowledge. More detailed analysis can be referred to DouZero [18]. Owing to these advantages of DMC, DouZero chooses to adopt

this technique to train the model and achieves outstanding performance.

C. Implementation Details of DouZero System

As for the implementation of DouZero system, it makes use of self-play procedure, where the actors play games to generate data and the learner update the models using these collected samples. The input of the networks is composed of state features and action features. The state features represent the information known to the player. Information of card combinations, such as the current player's cards, in hand are in the form of matrices and there also exist some one-hot vectors that represent information about number in this game, such as the number of cards of other players, and the number of bombs played so far. As for the action feature, it encodes the set of legal moves under current state where each action is encoded with a one-hot 4×15 card vector. When making the decision, current state s and each of the legal moves a_i , which is gained according to the game rules, are input to the model and the corresponding value $Q(s, a_i)$ is output. Based on these Q-values, an optimal action can be taken. As for the specific network architecture, a layer of long short-term memory (LSTM) takes the responsibility to process historical moves and this information is concatenated with other features, thus forming the global state. Also, there are six layers of MLP with a hidden size of 512 to output Q values, which serves as the main body of models that makes decision.

Besides, to improve the training efficiency, the system is implemented in a parallel way with multiple actors and one learner. Three global networks are maintained in the learner process for the three players and updated to approximate the target values based on data produced by actors. Similarly, each actor keeps three local networks which are synchronized with the global ones periodically. The learner and actors communicate through three shared experience buffers. In this way, the system can be trained in an effective self-play procedure.

IV. PROPOSED METHOD

After describing the basic rules of *DouDizhu* as well as the main algorithm and details about DouZero, in this section, we introduce opponent modeling, coach-guided training, and MC simulation in our design. The former two components are aimed at the card-playing phase and we can get our DouZero+ AI system by adopting these two techniques. As for MC simulation, this technique is used for the training of our bidding network, aimed for dealing with bidding phase of *DouDizhu*. After introducing this technique to train a bidding network to DouZero+, our full DouZero+, which can be applied to real-world games that includes both bidding phase and card-playing phase, is obtained.

A. Opponent Modeling

Opponent modeling studies methods to build models, which make predictions about various properties of the modeled agents, e.g., actions, goals, and so on. Classic methods, such as policy

¹[Online]. Available: https://en.wikipedia.org/wiki/Dou_dizhu.

reconstruction [38] and plan recognition [39], tend to develop parametric models for agent behaviors. In order to simplify the modeling process, these methods decouple the interactions between the modeled agent and others, which may introduce bias when the agents are coupled with each other during the interaction. In this way, a natural practice is to execute opponent modeling when concurrently training all the agents in a self-play procedure [40] and this implementation is suitable to the training of *DouDizhu* AI system. What is more, concurrent learning helps opponent modeling adapt to different levels of the agent as the evolution of the agent's skills during training can be witnessed.

According to the rule of *DouDizhu*, what cards the three players have played is public information, plus that a standard pack of cards is played in this game, knowing the hand of the next player is enough to grasp what cards all players have. To this end, we just predict the hand of the next player for simplicity when adopting opponent modeling in *DouDizhu*. As for the implementation, we can naturally take advantage of deep neural networks to make predictions. For clear description, we name the part of opponent modeling as the “prediction model” and the part that makes decisions as the “decision model.” Considering that DouZero trains three models for the three players in the game, we also train three prediction models for opponent modeling. The prediction model can be viewed as a multihead classifier and outputs the probability distribution of the number of each card in the hand of the next agent. To be specific, it has to predict how many Card 3, how many Card 4, etc., that next player has in hand. To be noted, the prediction of card distribution of next agent also implies knowing that for the other agent, since there are three agents only. Since we can realize the environment of *DouDizhu* by ourselves instead of using a black-box game core, we can acquire the true hand of the next player and use this information as ground truth to train the prediction model. What is more, taking Card 3 as an example, how many Card 3 the next player has at most can also be inferred from state information, according to current agent's hand and how many Card 3 has been played. We call this information “legal label” and this information can be utilized as a mask to prevent the prediction model from making impossible predictions.

When it comes to the implementation, the input of the prediction models is the same as state features in DouZero and the structure is also similar to DouZero except for the final layer, which works as a multihead classifier. The details of prediction model is shown in Fig 2. This model is trained using cross-entropy loss function as opponent modeling in this scenario can be viewed as a multiclassification problem. As for the decision model, we concatenate the predicted results output by the prediction model as well as the original state features. With these information and action features as the input, the decision model outputs Q-values for all legal moves of current state and thus the optimal action can be chosen according to these results. The model structure is almost the same as DouZero except the dimension of the first layer in MLP is different due to the extra features of predicted results. To sum up, Fig. 3 demonstrates the framework that combines opponent modeling with DouZero.

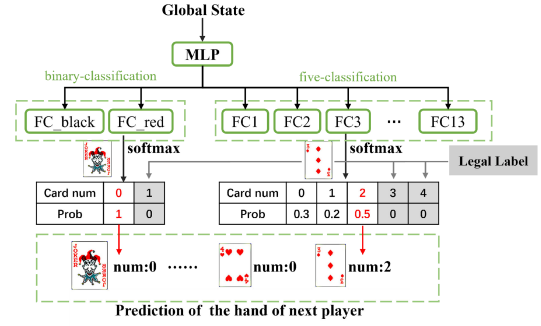


Fig. 2. Details of prediction model. It can be viewed as a multihead classifier, which mainly consists of five shared fully connected layers and a multihead fully connected (FC) layer to output the probability. We can extract “legal label” from the state information, which represents how many cards of each kind the next player has at most. An example is shown that the next player has at most two Card 3. This information can be used as a mask to prevent the prediction model from giving an impossible prediction that the next player has three or four Card 3 in his hand.

B. Coach-Guided Training

Because of the large state and action space of *DouDizhu*, although multiple parallel actors can improve the efficiency of sample generation, the training process still costs a lot of time. To this end, we propose a method to help the agent master the skills faster, which works in the card-playing phase. Under this setting, the initial hand cards of the three players can be regarded as being fixed at the beginning of the game. Whereas, *DouDizhu* is a shedding-type game where the player aims to empty his own hand cards before others, thus the result of the game highly depends on the quality of the initial hand cards. Specifically, if a very strong hand is dealt to a player at the beginning, he can easily win the game as long as no serious mistakes are made. In this way, such openings are of little value for learning as the agent can hardly learn new knowledge from these data. On the other hand, if one player always plays matches where the initial hand cards are relatively balanced, he can learn some skills faster and better as he will lose and receive a negative reward if he makes any unsuitable decision. In the setting of DouZero, the initial cards of the three players are generated randomly so that quite a few samples may be not matched in strength and their values for learning are limited. In these scenarios, playing games using these unbalanced openings may be a waste of the time to some degree. If the actors are only allowed to generate data that is based on balanced initial hand cards, the agent can learn faster and form policies that can deal with such situation.

Based on the above discussion, we put forward a “coach network” to measure whether an opening is balanced in strength. The input of this network is the initial hand cards of the three players, where the Landlord has 20 cards and each Peasant has 17 cards in hand. The network needs to predict the probability of winning for the Landlord in one game, which we call P_{win} . Then, we set a threshold, which is represented with β and it is smaller than 0.5. A too small P_{win} means that the strength of the Landlord's initial cards is so weak that he can hardly win the game and the reverse holds true as well. In this way, there is no

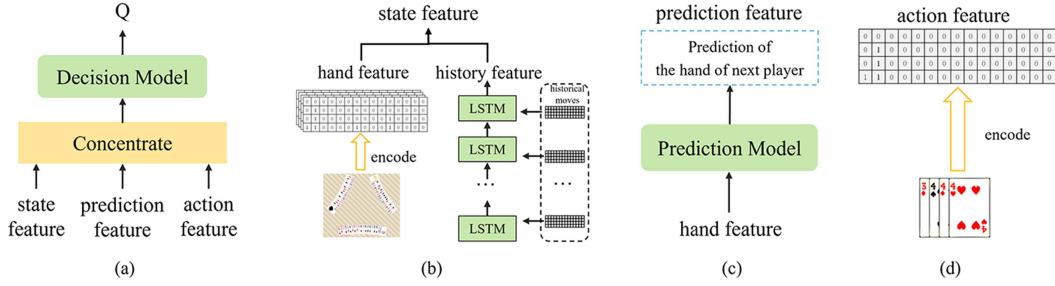


Fig. 3. Overview of the framework that introducing opponent modeling to DouZero and how different features are generated. The prediction model takes the state information as input and outputs the predicted number of every card in the hand of the next agent. The prediction result is concatenated with the state and action features and then all these information is input to the decision model to decide which action to take. The decision model is trained using a DMC algorithm like DouZero. To be mentioned, both decision models and prediction models include a layer of LSTM to process historical moves, which is shown in (b). (a) Overall framework. (b) State feature. (c) Prediction feature. (d) Action feature.

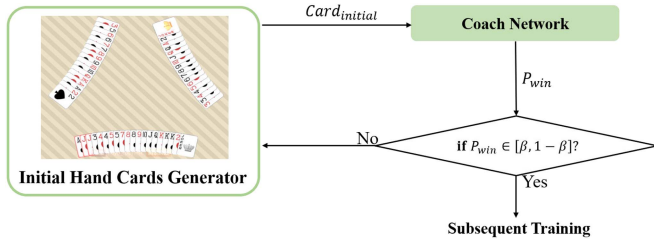


Fig. 4. Overview of the framework that utilizes coach-guided training. In this figure, $Card_{initial}$, P_{win} , and β represent generated initial hand cards of all players, the predicted probability of winning for Landlord and the threshold value, respectively. The coach network includes one embedding layer to gain representation vectors for the cards and several fully connected layers to make predictions. The model takes $Card_{initial}$ as input and outputs P_{win} . If P_{win} is in the defined range, i.e., larger than β and smaller than $1 - \beta$, the game with such $Card_{initial}$ will be continued, thus generating samples for training. Otherwise, this opening will be abandoned and another initial hand cards will be generated until an appropriate opening is obtained.

need for the actors to play games with unbalanced openings, thus setting aside time to carry on more valuable matches. The overall framework of our coach-guided learning is shown in Fig. 4. To be noted, to leave time for agents to learn necessary skills that are enough to deal with unbalanced matches, the threshold β is set to 0 at first and increases as the training goes on.

As for the architecture of the coach network, it consists of an embedding layer to process the input vectors and seven layers of fully connected layers with a hidden size of 128 to extract representations and make predictions. The model is trained using BCE loss as it can be viewed as a binary classifier. Similar to opponent modeling, the coach network is also concurrently trained with the decision models and we can use the results of self-play games as labels for the training. Whereas, as this module has a “God’s eye view” of the opening, only one coach network is needed, unlike DouZero where one player corresponds to one model. What is more, our coach network only works at the beginning of one game and does not influence the subsequent processes. Therefore, such an idea can also be transferred into the development of other similar game AIs and facilitates the training.

C. MC Simulation

In the bidding phase of *DouDizhu*, a player only knows his own 17 cards and has to make decisions based on such mere

information so how to measure the strength of these initial cards is very important. Intuitively, heuristic rules can be adopted in this case. For instance, hand cards with both red Joker and black Joker can be thought of as possessing certain strengths. However, the combination of 17 cards is very complex and heuristic rules can only help in very limited cases. Besides, what cards other players have is also unknown, making the measurement full of uncertainty and relying on heuristic rules impractical. In this way, training a model to give an accurate measurement no matter what cards are got seems to be a solution. In DouZero [18], it mentions that a bidding network is trained using human expert data and supervised learning. Unfortunately, this method is not that easy to put into practice due to the difficulty of accessing enough human data. To this end, we put forward an approach to train the bidding network, which is completely independent of human knowledge. We note that MC simulation is good at the estimating uncertainty, especially in estimation of the output of complex measurement systems [41] so that this approach can meet our requirement.

Before the training of the bidding network, we need to obtain a trained model, which is designed for the card-playing phase. For convenience, we call the trained model as a card-playing model and use this model to execute simulation. When applying MC simulation to measure the strength of a dealt hand, which is called C and consists of 17 cards, we fix C and shuffle other cards many times so that one case corresponds to one different opening. Then, we add the three Landlord cards to C , leading to 20 cards for the Landlord. After that, we can launch tournaments using the card-playing models and these generated openings of C . After enough games, we can calculate the average win ratio of the Landlord and this can be viewed as the strength of initial cards C . The whole process conforms to MC simulation as we approximate the strength using random sampling and the result is also of high confidence as long as the number of simulations is big enough. Repeat this process and we can thus obtain measurements of many different initial cards. In this way, we can acquire enough data and then supervised learning can be used to train a bidding network. Compared with DouZero, our training does not include human data so it is easy to implement. In addition, our technique can get an objective measurement of cards because human data may contain samples where the decision is unreasonable.

The architecture of the bidding network is similar to the coach network, which is made up of an embedding layer and five fully connected layers with a hidden size of 128. The prediction of the win ratio for a dealt hand is ranged from 0 to 1 so this is a regression problem and an mse loss function is used for the update. When implemented in real-world games, there are usually three kinds of risk stakes 1, 2, and 3, with 1 being the lowest and 3 being the highest. With the output of the trained bidding network, we can utilize grid search to investigate, which threshold values are appropriate and then our bidding network can be applied in real-world scenarios straightly.

V. EXPERIMENTS

In this section, we conduct experiments to demonstrate the performance of our *DouDizhu* AI system. Similar to the preliminary of this work, we evaluate the performance of opponent modeling and coach-guided training, respectively, and then combine them. After that, experiment results about the bidding network are shown.

A. Experiment Settings

In the evaluation of the our methods, we follow the practice of DouZero [18] and our preliminary work [24] to launch tournaments that include both Landlord and Peasants. Specifically, to compare two algorithms A and B , we first assign them to play as Landlord and Peasants, respectively, for a given deck and switch the sides, i.e., A plays the role of Peasants and B acts as Landlord using the same deck again. To show the performance of the model in the training process, the test is executed by playing 10 000 randomly sampled decks every 30 min. Considering that DouZero is the state-of-the-art *DouDizhu* AI and our AI system is based on it, we just compare the performance between them although there exist other AI programs.

When comparing the performance, we adopt the public fully trained models of DouZero as the baseline models, which we call “douzero(base).” To demonstrate the improvement of our AI, we also train original DouZero to intuitively exhibit the performance difference, namely, two AI systems are compared with “douzero(base)” models in the training process and the difference of the performances can show the effectiveness of our methods. As for the training and evaluation metric given an algorithm A and the opponent B , we adopt winning percentage (WP), which indicate the number of games won by A divided by total number of games. Although we use another metric average difference in points (ADP), which is related to the winning scores of games, in our preliminary work, we uncover that filtering out unbalanced games with coach network may affect the capability of the agents to win higher scores as they may tend to employ a cautious policy instead of taking risks for higher points. Whereas, considering the improvement that the coach network brings, we think such tradeoff is accepted.

Our implementation is based on DouZero, which adopts TorchBeast framework [42] and the training schedules such as numbers of actors and hyperparameters are kept the same as default ones. In the comparison, both our models and the original DouZero are tested with baseline models trained with the metric

of WP. All experiments are conducted on a server with 4 Intel(R) Xeon(R) Gold 6252 CPU @ 2.10 GHz and GeForce RTX 2080Ti GPU.

B. Evaluation of Opponent Modeling

In this part, the effectiveness of introducing opponent modeling is demonstrated. Observing the state features of DouZero contain, we find that all public information is contained so that the information of other players’ cards is implicitly included. In ideal scenarios, if the learning capability of the models is strong, the agent can infer the cards of its opponents from known information. In this way, we first make a pre-experiment to investigate whether opponent modeling can facilitate learning. In the experiment, we add the cards of the next player to the state features directly. To more clearly show the advantages of knowing others’ cards, we do not utilize the self-play procedure. Instead, the models of “Hacker” possesses the ability to know the cards hold by the next player. This Landlord model is trained through gameplay against “douzero(base)” Peasant models to generate data and execute training. The Peasant models are also trained using the same methodology, i.e., playing games against “douzero(base)” Landlord model.

For fairness, the original DouZero is also trained in the same way but the features of the next player’s cards are filled with zero, which we call “Hacker(base).” The result of this pre-experiment is shown in Fig. 5(a) and it can be observed that knowing the cards of the next player indeed boosts the performance of agents, especially for Peasants. Interestingly, the improvement of Peasants is far more obvious than that of the Landlord. We assume that reason behind this phenomenon is that knowing others’ cards not only helps to take better moves but also facilitates cooperation. In contrast, the Landlord operates independently without the benefits of collaboration so knowing the cards of others fails to give much room for improvement particularly when his hand cards are weak. In summary, the results of this experiment illustrate that introducing explicit information about others’ cards can indeed improve the performance of *DouDizhu* AI.

After validating our idea, we conduct experiments about opponent modeling following what is discussed in Section IV-A, that is, concurrently train prediction models as well as decision models. The results in Fig. 5(b) reveal that introducing opponent modeling mainly improves the performance of models for Peasants, consistent with the analysis above. Because the model has to take more features as input, introducing opponent modeling may slow down the learning at first but after enough training, the models manage to grasp more knowledge and achieve better performance.

In order to investigate more about the accuracy of opponent modeling, we also utilize the intersection over union (IOU) value as a metric. This value over a batch of data is computed as below:

$$\text{IOU} = \frac{\sum_B \min(V_{\text{pred}}, V_{\text{truth}})}{\sum_B \max(V_{\text{pred}}, V_{\text{truth}})} \quad (1)$$

where V_{truth} is the actual number of one kind of card, V_{pred} is the predicted number, and $\sum_B$ represents that the results are

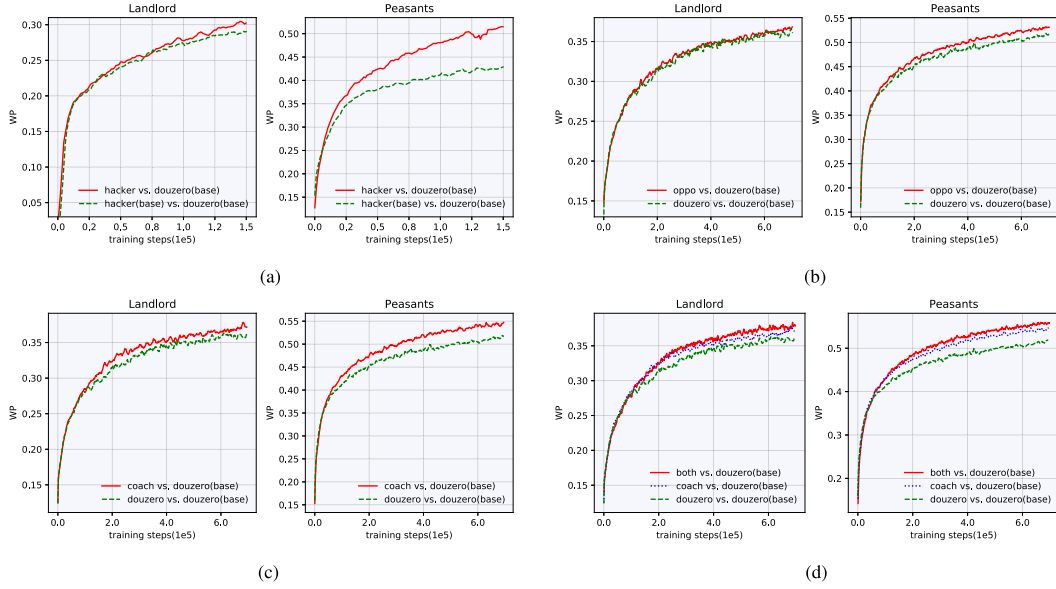


Fig. 5. WP of models adopting different components of our method and DouZero models. “Landlord” means that the models play as Landlord against Peasants of pretrained models provided by DouZero, which we call “douzero(base)” in the figure, and the same goes for the reverse. One training step means one update of networks in learner. (a) Hacker versus Hacker(base). “Hacker” means that models see the hand cards of the next player and “Hacker(base)” denotes the douzero models that are trained following the way how “Hacker” models are trained. (b) Opponent Modeling versus DouZero. “Opponent Modeling” refers to models utilizing opponent modeling. (c) Coach Network versus DouZero. “Coach Network” represents models combined with coach-guided training and the final threshold value of coach network is 0.3. (d) Both Improvements versus DouZero. Our model use both opponent modeling and coach-guided training. For clear comparison, the result of models improved by coach-guided training is also included.

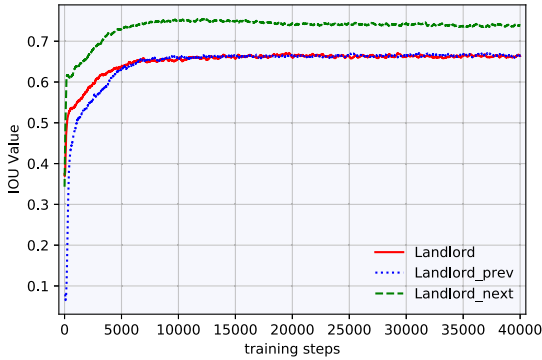


Fig. 6. Curves of IOU value for the prediction models when combining opponent modeling with DouZero. In the figure, the legends represent the positions of the models. For instance, “Landlord” means the accuracy of landlord model’s predicted results for the hand cards of its next player, i.e., Landlord_next.

obtained by summing over a batch of data. For instance, in one data sample, the next player has cards 334 555 in hand and the predicted result is 333 455. That is to say, the intersection set of the actual and predicted results is 33 455 while the union of these values is 3 334 555. That is to say, the intersection set of the actual and predicted results is 33 455 while the union of these values is 3 334 555. Using the equation, we can know that the IOU value is equal to $5/7$. In a few words, the closer the IOU value is to 1, the more accurate the predicted results are. We present the IOU value during the training of opponent modeling in Fig. 6. It can be observed that the IOU values for the prediction models at different positions are about 0.7. Considering the difficulty of predicting cards and the complexity of this game, this result is understandable, which may account for

the inobvious improvement of combining opponent modeling and DouZero.

In addition, the accuracy of “Landlord_next” model is relatively higher compared with the other two models according to the results. Because each agent is tasked with predicting the hand cards of the next player and players in *DouDiZhu* game take actions following a special order: “Landlord,” “Landlord_next,” and “Landlord_prev,” we can know that only the “Landlord_next” agent predicts the hand cards of its teammate, whereas the other two agents have to predict the cards of their opponents. We assume that the cooperative dynamics inherent in the Peasant agents’ strategy may establish a commonality in their behavior patterns, which in turn facilitates the task of predicting the hand cards of a teammate. In comparison, predicting the cards of an opponent is relatively more challenging. To this end, it is understandable that the prediction model for the “Landlord_next” agent achieves the best performance.

C. Evaluation of Coach-Guided Training

In this part, we show how the coach network performs in our AI system and the training procedure is introduced in Section IV-B. Through grid search about the hyperparameter β , the final threshold value starts from 0, increases by 0.05 for every 20 000 training steps, i.e., 20 000 updates of models in the learner, and its upper limit is set to 0.3. In Fig. 5(c), we presents the significant improvement that our method brings due to its effectiveness of this method. It can also be observed that the improvement of Peasants is more obvious than that of Landlord as Peasants can learn more skills and achieve greater advantages through cooperation in a balanced game. To show the

TABLE I
WP OF MODELS USING COACH-GUIDED TRAINING WITH DIFFERENT THRESHOLD VALUES

A \ B	Coach_0.20	Coach_0.25	Coach_0.30	Coach_0.35
Coach_0.20	-	0.5003 $\pm$ 0.0011	0.5087 $\pm$ 0.0009	0.5005 $\pm$ 0.0012
Coach_0.25	0.4997 $\pm$ 0.0011	-	0.5020 $\pm$ 0.0014	0.5006 $\pm$ 0.0015
Coach_0.30	0.4913 $\pm$ 0.0009	0.4980 $\pm$ 0.0014	-	0.5 $\pm$ 0.0014
Coach_0.35	0.4995 $\pm$ 0.0012	0.4994 $\pm$ 0.0015	0.5 $\pm$ 0.0014	-

These models are tested by playing 10 000 randomly sampled decks with each other. For stochastic meaning, we repeat the evaluation on five different random seeds. As for each result, the former value is average value while the latter one is the standard error. The value in table is computed by averaging the performance of landlord and peasants. model A outperforms model B if the corresponding value is larger than 0.5.

effect of threshold value on the performance, we also exhibit the performance of models with different β . Because their performances are close, drawing the curves of performance in a figure is difficult to differentiate their difference. In this way, we pick the models, which are both trained for 70 000 training steps and launch tournaments between them to evaluate their performance. The results are given in Table I and it can be observed that using a coach network whose upper limit is 0.3 makes the model achieve the best performance. To be mentioned, considering stochastic meaning, the evaluation is repeated on five different random seeds and we report the average values and standard errors. The small error indicates that 10 000 randomly sampled decks are enough to measure the performance of *DouDizhu* models.

What is more, similar to the preliminary work, some cases of the prediction result of our coach network are listed in Table II. The games are chosen from Botzone platform [20], [21], [22] and played between our model and other bots. On the platform, the evaluation is similar to the tournament introduced above in that the same deck is played between two AI programs twice. In case 1, it can be observed that the Landlord is allocated with a very strong hand, which consists of a Rocket Bomb and most cards of low rank that can compose combinations of Chain so that the Landlord can win the game easily so that it can be observed that the Landlord wins in the two games using this deck. As for case 2, the hand of the Landlord is weak as he has few cards of high rank. In this way, the Landlord can hardly win the games of this deck. In case 3, the initial hand cards are relatively balanced. At this time, the result of the game heavily depends on the capability of the agents. From the results, we can see that both our Landlord and Peasant models win the game by making appropriate decisions, reflecting the strength of our AI program and its capability to deal with balanced games. In a word, the strength of the model can hardly determine the result of the game if an unbalanced hand is dealt, while it plays an important role in a balanced game. These examples illustrate the importance of balanced samples and validate the idea of coach-guided learning, which helps the model to focus more on balanced games and grasp more useful skills.

D. Evaluation of Card-Playing Phase

From above discussion, both our proposed components can improve the performance of AI program in card-playing phase. We present the results of combining both opponent modeling and coach-guided training in Fig. 5(d). Considering that introducing coach network brings more obvious improvement than opponent

modeling, to intuitively demonstrate the effect of combining these two methods, the result of just using coach-guided training is also included in the figure. It can be observed that the performance of combining two methods is a little worse than merely utilizing coach-guided training at first, which is consistent with the discussion of just introducing opponent modeling. Considering the difficulty of making progress when the strength of models reach a certain level, even if the improvement is not that obvious, the results are also acceptable.

The experiment results shown above demonstrate that our methods help improve the performance of DouZero AI system. To verify whether our final model manages to outperform DouZero, we train the models for enough steps and the results are exhibited in Table III. We test the performance of our DouZero+, which is trained for 5e9 frames and each update involves 3200 frames, and the original DouZero trained for 1e10 frames against the public DouZero baseline model. It can be observed that the public baseline achieves a better performance than DouZero training for 1e10 frames in both roles, indicating that the baseline model is trained for more steps. However, our DouZero+ succeeds in beating it using less than half training frames, revealing the strength of our model and the effectiveness of our methods. In addition, we can know that keep training our DouZero+ can further enhance the performance of our model, but the improvement is not that obvious. We think it understandable as achieving substantial progress becomes increasingly challenging when the model already demonstrates a high level of performance.

E. Evaluation of Bidding Phase

In addition to the improvements we made in the card-playing phase, we also make experiments to evaluate the bidding network, which is trained following what Section IV-C describes. In the implementation, we use about two hundred thousand samples to train our model and the strength for each initially dealt hand, which consists of 17 cards, is ranged from 0 to 1. In determining the number of simulations for each sample, we conduct a pre-experiment by generating 1000 samples, which are simulated 1000 to 10 000 times. We uncover that the results are very stable between different numbers of simulations as the maximal standard error is only about 0.015. To this end, we choose to generate 5000 games to simulate the strength of each sample, considering the time consumption and accuracy of the simulation.

As the bidding network of DouZero is not public, to evaluate the performance of our model, we conduct an experiment by comparing with our designed two bidding networks. For better understanding, we first introduce the environment settings of the experiment. In a *DouDizhu* game, there exist three risk stakes in *DouDizhu*, which also determine the base score of the game. For instance, if one player becomes the Landlord through raising bidding of 2 and finally wins the game, he will get four points and other players will lose two points. Besides, every time a bomb is played in the game, the base point will double. In the bidding phase, the players have to make bids in turn and a player can only accept the prior player's bid by passing his turn or overbid the previous player as long as the prior one did not bet 3 as the

TABLE II
CASE STUDY TO SHOW THE EFFECT OF COACH NETWORK

	Landlord	Landlord_next	Landlord_prev	Predicted P_{win}	Our result (Landlord)	Our result (Peasants)
Case1	344566777889TJQQ22BR	333556789TJQQKAA2	4456899TJJKKKAA2	0.8735	Win	Lose
Case2	35556667789TJJKA22B	3346778889JJQQA	344459TTQQKKKA22R	0.2277	Lose	Win
Case3	3445558889TTQKKAAA2R	3336667999JQKK22B	44567778TTJJJQQA2	0.5310	Win	Win

It predicts the winning probability of landlord based on the initial hand cards of the three players. We pick some cases from games between our model and other bot on Botzone to show the predicted results of coach network and the actual result from the view of the our model. The role in the brackets represent the role that our model play. To be mentioned, T means 10, J means Jack, Q means Queen, K means King, A means Ace, B means black Joker, and R means red Joker.

TABLE III
WP OF OUR DOUDIZHU AI SYSTEM AGAINST DOUZERO PUBLIC MODEL BY PLAYING 10000 RANDOMLY SAMPLED DECKS

	A	DouZero+ (1e10)		DouZero+ (5e9)	
		L	P	L	P
douzero(public)		0.4104 ± 0.0004	0.6217 ± 0.0002	0.4063 ± 0.0007	0.6026 ± 0.0006
	A	DouZero (1e10)		DouZero (public)	
		L	P	L	P
douzero(public)		0.3992 ± 0.0008	0.5756 ± 0.0011	0.4061 ± 0.0005	0.5939 ± 0.0005

L: WP of a as Landlord; P: WP of a as Peasants. The numbers in the brackets represent the training frames of the corresponding models. The evaluation is repeated on five different random seeds. The former value of each result is the average value while the latter one is the standard error.

risk stake. If one player makes bidding of 3, he will straightly become the Landlord, otherwise, the highest bidder will take the Landlord position. If all players refuse to offer a bid, player 1 will become the Landlord automatically in our setting. After then, the game enters the card-playing phase and all players adopt our models aimed at card playing.

As discussed above, there are usually three risk stakes in *DouDizhu* games. We need to set three threshold values to determine the appropriate bidding decision based on the strength measurement obtained from the trained bidding network. The bidding decision satisfies the following equation:

$$\text{bidding} = \begin{cases} 0, & 0 \leq v < \text{value1} \\ 1, & \text{value1} \leq v < \text{value2} \\ 2, & \text{value2} \leq v < \text{value3} \\ 3, & \text{value3} \leq v \leq 1 \end{cases} \quad (2)$$

where v is the output of the bidding network, which is normalized between 0 and 1, and the three values are the threshold values to be set, which can be realized by grid search. Considering an effective bidding strategy contributes to improved performance as a Landlord, we explore different sets of threshold values and record the average WP and ADP for the Landlord when conducting grid search. To be noted, we can take bidding information of others into consideration during the grid search. For instance, when player 1 makes a bid of 2, the threshold values for player 2 can be adjusted accordingly compared with the scenario where player 1 does not offer a bid, as player 1 is more likely to have good hand cards in the former condition. Analogically, we can set different threshold values based on the positions of the players and bidding information of others. As considering so many cases may involve too many hyperparameters, we just set a uniform set of threshold values for simplicity. In addition, since the setting of threshold value is not directly related to the key idea of our algorithm, we omit the specific experiment process in the manuscript and just report the final result: value1—0.3, value2—0.425, and value3—0.575.

TABLE IV
PERFORMANCE OF ADOPTING DIFFERENT BIDDING METHODS

ADP	Method	Position		
		B	Q	R
	Player 1	0.385 ± 0.019	0.290 ± 0.027	-0.618 ± 0.026
	Player 2	0.308 ± 0.030	0.128 ± 0.013	-0.559 ± 0.027
	Player 3	0.230 ± 0.017	0.175 ± 0.022	-0.362 ± 0.026

All these results are obtained using the same randomly generated 10 000 decks and we reserve three decimal places for the results. In the method row, “B” represents that this player adopt our bidding network, “R” means that this player utilize random bidding, and “Q” represents that the player use the model trained by Q-learning algorithms. “ADP” is used as the metric to measure the performance and we list the averaged results for each algorithm at each position. We repeat the evaluation on five different random seeds. The former value of each result is the average value while the latter one is the standard error.

As for our designed baseline bidding networks, the first one adopts random bidding, i.e., players who adopt this method make bids based on a random digit, which complies with the rules, instead of the estimation of their hand cards. The other method is referred to [43], which introduces reinforcement Q-Learning to Bridge bidding. We call this method Q-Learning-based bidding. In this method, a neural network is adopted to predict the state-action value for the initial hand cards and each possible bidding action. The input of the network contains the initially dealt 17 cards and the bidding decision made by the agent’s preceding player. The network functions as a multihead classifier and outputs Q-values of different bidding decisions for one agent so that we can select appropriate bidding decision according to the output. After determining the Landlord, we adopt the trained card-playing models to play the subsequent game and generate the game’s outcome. To be noted, for each agent that has made a bid (some agent may not place a bid as its previous player makes a bid of 3), we record its input, output, and score as the data sample for subsequent learning. In the context of *DouDizhu*, the bidding process for each agent can be considered as a one-step decision problem, and we can adopt Q-Learning technique to execute the training.

As there are three players in a *DouDizhu* game, quite a few different cases for using these three algorithms exist considering the full permutation. We report the averaged performance for each algorithm at each position, namely, each listed result is averaged over nine different cases. In addition, “ADP” is used as the metric as bidding network mainly helps a player to win high scores. The experimental results are presented in Table IV. It can be observed that there exists apparent improvement when adopting our bidding network as players utilizing our method in the bidding phase obviously outperform those who adopt other two baseline bidding methods across all model positions. In the Q-Learning-based method, the trained model only predicts

Figure 7 consists of two screenshots of the Botzone platform leaderboard for the game *FightTheLandlord*. Both screenshots show a table with columns: Rank, Bot Name, Owner, Score, Description, and Version. In both cases, the top-ranked bot is 'test' owned by 'loyd_cpp' with a score of 1607.37 (a) and 1414.24 (b). Other bots include 'Dou1Dou', 'version2', 'douzero_community', and 'dqj'.

(a)

(b)

Fig. 7. Screenshots of our ranks on Botzone platform for our two *DouDizhu* AI systems at 19 October, 2022. (a) *DouZero+* and (b) full *DouZero+*, which is equipped with a bidding network. For clear understanding, we replace contents in title with English. Our bots both rank top among 428 and 268 AI programs, respectively. The link for Botzone is <https://botzone.org.cn/>.

values of a player's initial hand cards while the other two players' cards are unknown. However, different opponents' hand cards may lead to completely different outcome of a game, resulting in the instability of the training and hindering the performance of this method. In contrast, our method uses MC simulation to obtain a more objective evaluation of an initial hand so that our bidding network shows an outstanding performance. Furthermore, since we just adopt a uniform set of threshold values without considering the bidding information of other agents, there is still potential for further improvement in the performance of our method.

It is also interesting that the three players seem to be not equivalent in the game as our bidding network and Q-Learning-based bidding both achieve a higher score in the first place while random bidding performs the opposite. We assume that this is because having priority to make the bidding first gives a reasonable player more initiative. Whereas, making a bidding in an unreasonable way may make the player more unfavorable when he has the priority so that random bidding performs relatively well when it is adopted in a rear position. It is these characteristics that make the bidding of *DouDizhu* game interesting and challenging, demonstrating the meaning of our work to a certain degree.

F. Performance on Botzone Platform

Finally, to more comprehensively test our AI, we also upload models to Botzone [22], an online platform with hundreds of AI programs whose play styles are diverse. This platform launches tournaments between bots, in a way similar to our experiments. According to the results of games, Botzone maintains a leaderboard by Elo rating scores and the rank can reflect the strength of a bot.

There are two versions of the *DouDizhu* game supported by the platform, namely, *Fight The Landlord* and *Fight The Landlord 2*. The former one only contains the card-playing phase while the other comprises both the bidding phase and card-playing phase and each version maintains its own leaderboard. We upload our AI systems to *Fight The Landlord* and *Fight The Landlord 2* on the platform, respectively, and they manage to rank the first, among more 400 and 250 AI programs. Fig. 7 exhibits the rank of our two AI system on the corresponding Botzone leaderboard. The screenshots were taken in October, 2022, but our AI bots keep the rank until now. The results demonstrate that our AI is not only strong enough, but also able to deal with opponents of different styles.

VI. CONCLUSION

In this article, we put forward a stronger *DouDizhu* AI system than the state-of-the-art AI, *DouZero*. Inspired by the human player's prediction about opponents' hand cards in practice and the characteristic of high variance for this game, our *DouZero+* utilizes opponent modeling and coach-guided learning to enhance the performance. Using MC simulation, our full version of *DouZero+* gets rid of the dependence on human expert data to train a bidding network, helping the agent to make reasonable decisions in the bidding phase and our system to be applicable to real-world scenarios. The outstanding performance of our AI system in Botzone community demonstrates the effectiveness of our method.

REFERENCES

- [1] D. Silver et al., "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [2] H. Baier, A. Sattaur, E. J. Powley, S. Devlin, J. Rollason, and P. I. Cowling, "Emulating human play in a leading mobile card game," *IEEE Trans. Games*, vol. 11, no. 4, pp. 386–395, Dec. 2019.
- [3] I. P. Pinto and L. R. Coutinho, "Hierarchical reinforcement learning with Monte Carlo tree search in computer fighting game," *IEEE Trans. Games*, vol. 11, no. 3, pp. 290–295, Sep. 2019.
- [4] D. Perez-Liebana, J. Liu, A. Khalifa, R. D. Gaina, J. Togelius, and S. M. Lucas, "General video game AI: A multitask framework for evaluating agents, games, and content generation algorithms," *IEEE Trans. Games*, vol. 11, no. 3, pp. 195–214, Sep. 2019.
- [5] D. Silver et al., "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [6] D. Silver et al., "A general reinforcement learning algorithm that masters chess, Shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
- [7] J. Heinrich and D. Silver, "Deep reinforcement learning from self-play in imperfect-information games," in *Proc. 17th Int. Conf. Auton. Agents MultiAgent Syst.*, 2016.
- [8] M. Moravik et al., "Deepstack: Expert-level artificial intelligence in heads-up no-limit poker," *Science*, vol. 356, no. 6337, pp. 508–513, 2017.
- [9] N. Brown and T. Sandholm, "Superhuman ai for heads-up no-limit poker: Libratus beats top professionals," *Science*, vol. 359, no. 6374, pp. 418–424, 2018.
- [10] J. Li et al., "Suphx: Mastering Mahjong with deep reinforcement learning," 2020, *arXiv:2003.13590*.
- [11] O. Vinyals et al., "Grandmaster level in Starcraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [12] C. Berner et al., "Dota 2 with large scale deep reinforcement learning," 2019, *arXiv:1912.06680*.
- [13] D. Ye et al., "Mastering complex control in MOBA games with deep reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 6672–6679.
- [14] T. W. Neller and M. Lanctot, "An introduction to counterfactual regret minimization," *Educ. Adv. Artif. Intell.*, vol. 11, 2013.
- [15] H. Fu et al., "Actor-critic policy optimization in a large-scale imperfect-information game," in *Proc. Int. Conf. Learn. Representations*, 2021.

- [16] Y. You, L. Li, B. Guo, W. Wang, and C. Lu, "Combinatorial q-learning for dou di zhu," in *Proc. AAAI Conf. Artif. Intell. Interactive Digit. Entertainment*, vol. 16, no. 1, 2020, pp. 301–307.
- [17] Q. Jiang, K. Li, B. Du, H. Chen, and H. Fang, "Deltadou: Expert-level Doudizhu AI through self-play," in *Proc. Int. Joint Conferences Artif. Intell.*, 2019, pp. 1265–1271.
- [18] D. Zha et al., "Douzero: Mastering Doudizhu with self-play deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12333–12344.
- [19] R. S. Sutton and A. G. Barto, "Reinforcement learning: An introduction," Cambridge, MA, USA: MIT Press, 2018.
- [20] H. Zhang, G. Gao, W. Li, C. Zhong, W. Yu, and C. Wang, "Botzone: A game playing system for artificial intelligence education," in *Proc. Int. Conf. Front. Educ.: Comput. Sci. Comput. Eng. (FECS)*, 2012.
- [21] H. Zhou, Y. Zhou, H. Zhang, H. Huang, and W. Li, "Botzone: A competitive and interactive platform for game ai education," in *Proc. ACM Turing Celebration Conf.-China*, 2017, pp. 1–5.
- [22] H. Zhou, H. Zhang, Y. Zhou, X. Wang, and W. Li, "Botzone: An online multi-agent competitive platform for ai education," in *Proc. ACM Conf. Innov. Technol. Comput. Sci. Educ.*, 2018, pp. 33–38.
- [23] W. Li et al., "Teaching AI algorithms with games including Mahjong and Fight The Landlord on the botzone online platform," in *Proc. ACM Conf. Glob. Comput. Educ.*, 2019, pp. 129–135.
- [24] Y. Zhao, J. Zhao, X. Hu, W. Zhou, and H. Li, "Douzero : Improving Dou Dizhu AI by opponent modeling and coach-guided learning," in *Proc. IEEE Conf. Games*, 2022, pp. 127–134.
- [25] M. Buro, J. R. Long, T. Furtak, and N. Sturtevant, "Improving state evaluation, inference, and search in trick-based card games," in *Proc. Int. Joint Conf. Artif. Intell.*, 2009, pp. 1407–1413.
- [26] M. Gedda, M. Z. Lagerkvist, and M. Butler, "Monte Carlo methods for the game Kingdomino," in *Proc. IEEE Conf. Comput. Intell. Games*, 2018, pp. 1–8.
- [27] J. Zhou, "Design and application of tibetan long chess using Monte Carlo algorithm and artificial intelligence," in *Proc. J. Physics: Conf. Ser. (JPCS)*, vol. 1952, no. 4, 2021, Art. no. 0 42104.
- [28] F. Southey et al., "Bayes' bluff: Opponent modelling in poker," 2012, *arXiv:1207.1411*.
- [29] H. He, J. Boyd-Graber, K. Kwok, and H. Daumé III, "Opponent modeling in deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1804–1813.
- [30] S. J. Knegt, M. M. Dragan, and M. A. Wiering, "Opponent modelling in the game of tron using reinforcement learning," in *Proc. Int. Conf. Agents Artif. Intell.*, 2018, pp. 29–40.
- [31] J. Foerster, R. Y. Chen, M. Al-Shedivat, S. Whiteson, P. Abbeel, and I. Mordatch, "Learning with opponent-learning awareness," in *Proc. 17th Int. Conf. Auton. Agents and MultiAgent Syst.*, 2018, pp. 122–130.
- [32] C. Silva, R. O. Moraes, L. H. Lelis, and K. Gal, "Strategy generation for multi-unit real-time games via voting," *IEEE Trans. Games*, vol. 11, no. 4, pp. 426–435, Dec. 2019.
- [33] S. Raychaudhuri, "Introduction to Monte Carlo simulation," in *Proc. Winter Simul. Conf.*, 2008, pp. 91–100.
- [34] R. R. Trippi, "The Aim game: Learning investment management principles through Monte Carlo simulation," *Interfaces*, vol. 26, no. 3, pp. 66–76, 1996.
- [35] S. Kupferschmid and M. Helmert, "A Skat player based on Monte-Carlo simulation," in *Proc. Int. Conf. Comput. Games*, 2006, pp. 135–147.
- [36] J.-C. CHEN, S.-C. Tang, and I.-C. WU, "Monte-Carlo simulation for Mahjong," *J. Inf. Sci. Eng.*, vol. 38, no. 4, pp. 775–790, 2022.
- [37] N. Mizukami and Y. Tsuruoka, "Building a computer Mahjong player based on Monte Carlo simulation and opponent models," in *Proc. IEEE Conf. Comput. Intell. Games*, 2015, pp. 275–283.
- [38] D. Carmel and S. Markovitch, "Model-based learning of interaction strategies in multi-agent systems," *J. Exp. Theor. Artif. Intell.*, vol. 10, no. 3, pp. 309–332, 1998.
- [39] M. Fagan and P. Cunningham, "Case-based plan recognition in computer games," in *Proc. Int. Conf. Case-Based Reasoning*, 2003, pp. 161–170.
- [40] T. Bansal, J. Pachocki, S. Sidor, I. Sutskever, and I. Mordatch, "Emergent complexity via multi-agent competition," in *Proc. Int. Conf. Learn. Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=Sy0GnUxCb>
- [41] C. E. Papadopoulos and H. Yeung, "Uncertainty estimation and Monte Carlo simulation method," *Flow Meas. Instrum.*, vol. 12, no. 4, pp. 291–298, 2001.
- [42] H. Küttler et al., "Torchbeast: A pytorch platform for distributed RL," 2019, *arXiv:1910.03552*.
- [43] C.-K. Yeh, C.-Y. Hsieh, and H.-T. Lin, "Automatic bridge bidding using deep reinforcement learning," *IEEE Trans. Games*, vol. 10, no. 4, pp. 365–377, Dec. 2018.



Youpeng Zhao (Graduate Student Member, IEEE) received the B.E. degree in electronic engineering and information science in 2021 from the University of Science and Technology of China (USTC), Hefei, China, where he is currently working toward the M.E. degree in information and communication engineering with the Department of Electronic Engineering and Information Science.

His research interests include reinforcement learning and multi-agent systems.



Jian Zhao received the Ph.D. degree in electronic engineering and information science from the University of Science and Technology of China, Hefei, China, in 2023.

His research interests include reinforcement learning and multi-agent system.



Xunhan Hu received the M.E. degree in electronic engineering and information engineering from the University of Science and Technology of China (USTC), Hefei, China, in 2023.

Her research interests include reinforcement learning and multi-agent system.



Wengang Zhou (Senior Member, IEEE) received the B.E. degree in electronic information engineering from Wuhan University, Wuhan, China, in 2006, and the Ph.D. degree in electronic engineering and information science from University of Science and Technology of China (USTC), Hefei, China, in 2011.

From 2011 to 2013, he worked as a Postdoc Researcher with the Computer Science Department, University of Texas at San Antonio, San Antonio, TX, USA. He is currently a Professor with the EEIS Department, USTC his research interests, which include

multimedia information retrieval, computer vision, and computer game.

Dr. Zhou was the recipient of the Best Paper Award for ICIMCS 2012. He was the Publication Chair of IEEE ICME 2021.



Houqiang Li (Fellow, IEEE) received the B.S., M.Eng., and Ph.D. degrees in electronic engineering from the University of Science and Technology of China, Hefei, China, in 1992, 1997, and 2000, respectively.

He is currently a Professor with the Department of Electronic Engineering and Information Science. He has authored and coauthored over 200 papers in journals and conferences. His research interests include reinforcement learning, multimedia search, image/video analysis, and video coding and

communication.

Dr. Li is the winner of National Science Funds for Distinguished Young Scientists, the Distinguished Professor of Changjiang Scholars Program of China, and the Leading Scientist of Ten Thousand Talent Program of China. He is an Associate Editor (AE) for IEEE TRANSACTIONS ON MULTIMEDIA and was the AE of IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS FOR VIDEO TECHNOLOGY. He was the General Co-Chair of ICME 2021 and the TPC Co-Chair of VCIP 2010. He was the recipient of the Best Paper Award for ACM MUM in 2011, Best Paper Award for VCIP 2012, National Natural Science Award of China (second class) in 2015, and National Technological Invention Award of China (second class) in 2019.